Master of Computer Application

Semester– II
MACHINE LEARNING
THEORY AND PRACTICAL

PROJECT TITLE

PASSWORD STRENGTH DETECTION USING ML

Submitted By :

MATHEWS VARKEY - 2511022250005

Submitted To :

TANISHA DAS

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
March 2025

Password Strength Analysis Using Machine Learning

Appu415

1 Abstract

Passwords are a critical component of cybersecurity. This project presents a Machine Learning-based system that classifies passwords into Weak, Medium, and Strong categories. It also estimates crack time and provides feedback.

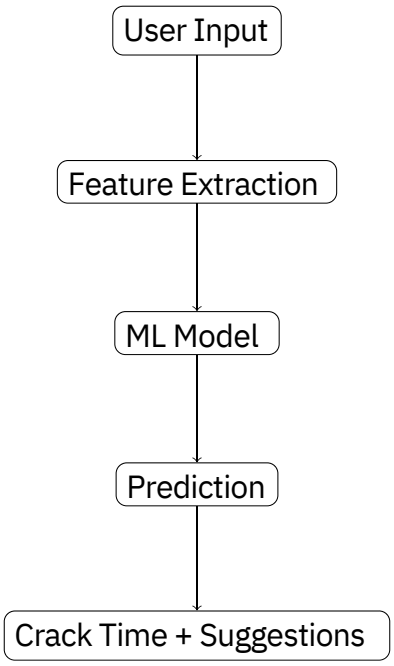
2 Intro duction

In modern digital systems, passwords are the first line of defense. However, weak passwords are easily compromised through brute force and dictionary attacks.

This project solves:

- Weak password detection
- User awareness improvement
- Automated strength classification

3 System Pipeline



4 DetailedExecutionFlow

ImportLibraries → Load Dataset → CleanData → Feature Extraction → Train/Test Split → ModelTraining → Evaluation → Prediction

5 DatasetDescription

The dataset consists of labeled passwords categorized into:

- Weak
- Medium
- Strong

Each password is analyzed based on structure and randomness.

6 Feature Engineering

Feature engineering is the backbone of this project.

6.1 Entropy Concept

Entropy measures randomness:

$$\text{Entropy} = \frac{\text{Unique Characters}}{\text{Total Length}}$$

Higher entropy means stronger password.

7 Feature Extraction Code

```
def extract_features(password):
    length = len(password)
    digits = sum(c.isdigit() for c in password)
    upper = sum(c.isupper() for c in passwor
    lower = sum(c.islower() for c in d)
    special = sum(not c.isalnum() for c in password)
    entrop = len(set(password)) / lengt
    y [length, digits, upper, h special, entropy]
    return lower,
```

Explanation: Each line extracts a specific property of the password.

8 ModelTraining

```
model= RandomForestClassifier()
model.fit(X_train, y_train)
```

Random Forest uses multiple decision trees to improve accuracy.

9 ModelEvaluation

9.1 Accuracy

Measures correct predictions.

9.2 Precision and Recall

Precision = Correct positives / Predicted positives

Recall = Correct positives / Actual positives

10 Prediction System

```
def predict_strength(password):
    features = extract_features(password)
    return model.predict([features])
```

This function processes user input and predicts strength.

11 Crack Time Estimation

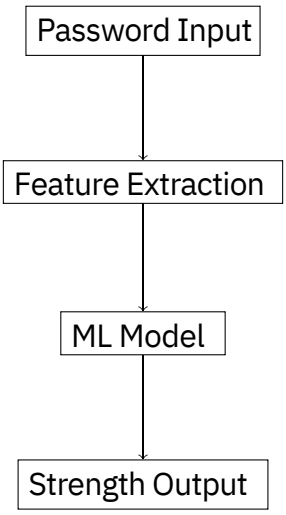
Crack time depends on:

- Password length
- Character complexity
- Search space

11.1 Mathematical Insight

$$\text{Time} \propto \text{CharacterSet}^{\text{Length}}$$

12 System Architecture



13 Advantages

- Improves security awareness
- Prevents weak password usage
- Easy to integrate

14 Limitations

- Depends on dataset quality
- Cannot detect all real-world attack patterns

15 Applications

- Banking systems
- Login authentication
- Cybersecurity tools

16 Future Enhancements

- Deep learning models
- Browser extensions
- AI password generator

17 Conclusion

This project demonstrates how Machine Learning enhances password security through intelligent prediction and analysis.

18 GitHubLink

https://github.com/appu415/PASSWORD-STRENGTH-DETECTION-USING-ML.git